New Tab | Home | Untitled | how to run selected tab in jupy

localhost:8888/notebooks/Untitled.ipynb

Common Recruitme... | Gmail | Adobe Acrobat | Kickass - Download... | Learning on Simplil... | SQL Practice Queries | SQL 50 - Study Plan... | Dashboard | iStudio | (112) Learn Python... | All Bookmarks

jupyter Untitled Last Checkpoint: 11 days ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab | Python 3 (ipykernel)

[3]:
```python
import csv
import os
from datetime import datetime

# --- Configuration ---
EXPENSES_FILE = "expenses.csv"

# --- In-memory storage ---
expenses = []  # list of dicts: {'date', 'category', 'amount', 'description'}
monthly_budget = None  # float or None

# --- Utilities & Validation ---


def validate_date(date_str: str) -> bool:
    """Return True if date_str matches YYYY-MM-DD."""
    try:
        datetime.strptime(date_str, "%Y-%m-%d")
        return True
    except (ValueError, TypeError):
        return False


def safe_parse_amount(amount_str: str):
    """Try to parse a string as float. Return (float, None) or (None, error_msg)."""
    try:
        amt = float(amount_str)
        if amt < 0:
            return None, "Amount cannot be negative."
        return amt, None
    except (ValueError, TypeError):
        return None, "Invalid number format."
```

[ ]:

[2]:

```python
# add expense
def add_expense():
    """Prompt user for expense details, validate them, and append to expenses list."""
    print("\nAdd an expense (enter 'q' at any prompt to cancel):")
    while True:
        date_input = input(" Date (YYYY-MM-DD): ").strip()
        if date_input.lower() == 'q':
            print("Add expense cancelled.\n")
            return
        if not validate_date(date_input):
            print("  → Invalid date format. Please use YYYY-MM-DD (e.g. 2024-09-18).")
            continue
        break

    while True:
        category = input(" Category (e.g. Food, Travel): ").strip()
        if category.lower() == 'q':
            print("Add expense cancelled.\n")
            return
        if not category:
            print("  → Category cannot be empty.")
            continue
        break
```

```python
while True:
    amount_input = input(" Amount (e.g. 15.50): ").strip()
    if amount_input.lower() == 'q':
        print("Add expense cancelled.\n")
        return
    amount, err = safe_parse_amount(amount_input)
    if err:
        print(f"  → {err}")
        continue
    break

while True:
    description = input(" Description (brief): ").strip()
    if description.lower() == 'q':
        print("Add expense cancelled.\n")
        return
    if not description:
        print("  → Description cannot be empty.")
        continue
    break

entry = {
    "date": date_input,
    "category": category,
    "amount": float(amount),
    "description": description,
}
expenses.append(entry)
```

```python
# view expense
def view_expenses():
    """Display all valid stored expenses. Skip or notify if incomplete entries found."""
    if not expenses:
        print("\nNo expenses recorded yet.\n")
        return

    print("\nStored expenses:")
    total = 0.0
    valid_count = 0
    skipped_count = 0

    for idx, e in enumerate(expenses, start=1):
        # Validate required fields
        date = e.get("date")
        category = e.get("category")
        amount = e.get("amount")
        description = e.get("description")

        malformed = False
        reasons = []
        if not date or not validate_date(date):
            malformed = True
            reasons.append("invalid/missing date")
        if not category:
            malformed = True
            reasons.append("missing category")
        try:
```

```python
if not date or not validate_date(date):
    malformed = True
    reasons.append("invalid/missing date")
if not category:
    malformed = True
    reasons.append("missing category")
try:
    amt = float(amount)
    if amt < 0:
        malformed = True
        reasons.append("negative amount")
except Exception:
    malformed = True
    reasons.append("invalid/missing amount")
if not description:
    malformed = True
    reasons.append("missing description")

if malformed:
    skipped_count += 1
    print(f" {idx}. [SKIPPED] Incomplete entry: {e}  → {', '.join(reasons)}")
    continue

# Print formatted valid entry
print(f" {idx}. {date} | {category} | {amount:.2f} | {description}")
total += float(amount)
valid_count += 1
```

```python
# calculate total expense
def calculate_total_expenses() -> float:
    """Return the sum of valid expense amounts."""
    total = 0.0
    for e in expenses:
        try:
            amt = float(e.get("amount", 0))
            if amt >= 0:
                total += amt
        except Exception:
            # skip malformed amounts
            pass
    return total
```

```
5]: #set monthaly budget
def set_monthly_budget():
    """Prompt the user to enter the monthly budget and store it."""
    global monthly_budget
    while True:
        b = input("Enter your monthly budget amount (e.g. 1000.00): ").strip()
        amt, err = safe_parse_amount(b)
        if err:
            print(f"  → {err}")
            continue
        monthly_budget = amt
        print(f"Monthly budget set to: {monthly_budget:.2f}\n")
        return
```

```python
# track budget
def track_budget():
    """
    Compare total expenses with user's monthly budget.
    - If budget not set, prompt to set one.
    - Show remaining balance or a warning if exceeded.
    """
    global monthly_budget
    if monthly_budget is None:
        print("\nNo monthly budget set yet.")
        set_now = input("Would you like to set it now? (y/n): ").strip().lower()
        if set_now == 'y':
            set_monthly_budget()
        else:
            print("Budget tracking cancelled.\n")
            return

    total = calculate_total_expenses()
    print(f"\nTotal expenses so far: {total:.2f}")
    print(f"Monthly budget: {monthly_budget:.2f}")

    if total > monthly_budget:
        diff = total - monthly_budget
        print(f"  You have exceeded your budget by {diff:.2f}!")
    elif total == monthly_budget:
        print("You have used your full budget for the month.")
    else:
        remaining = monthly_budget - total
        print(f"You have {remaining:.2f} left for the month.")
```

```python
# save and load csv
def save_expenses_to_csv(filename: str = EXPENSES_FILE):
    """Save all expenses to a CSV file (date, category, amount, description)."""
    try:
        with open(filename, mode="w", newline="", encoding="utf-8") as f:
            fieldnames = ["date", "category", "amount", "description"]
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            for e in expenses:
                # Ensure we write safe values
                row = {
                    "date": e.get("date", ""),
                    "category": e.get("category", ""),
                    "amount": f"{float(e.get('amount', 0)):.2f}" if e.get("amount") is not None else "",
                    "description": e.get("description", ""),
                }
                writer.writerow(row)
        print(f"Expenses saved to '{filename}'.\n")
    except Exception as exc:
        print(f"Error saving expenses: {exc}\n")
```

```python
def load_expenses_from_csv(filename: str = EXPENSES_FILE):
    """Load expenses from a CSV file into the in-memory list (validate each row)."""
    if not os.path.exists(filename):
        # No file to load
        return

    loaded = 0
    skipped = 0
    try:
        with open(filename, mode="r", newline="", encoding="utf-8") as f:
            reader = csv.DictReader(f)
            for row in reader:
                date = (row.get("date") or "").strip()
                cat = (row.get("category") or "").strip()
                amt_str = (row.get("amount") or "").strip()
                desc = (row.get("description") or "").strip()

                # Basic validation
                if not date or not validate_date(date):
                    skipped += 1
                    continue
```

```
            # Basic validation
            if not date or not validate_date(date):
                skipped += 1
                continue
            amt, err = safe_parse_amount(amt_str)
            if err:
                skipped += 1
                continue
            if not cat or not desc:
                skipped += 1
                continue

            expenses.append({
                "date": date,
                "category": cat,
                "amount": float(amt),
                "description": desc,
            })
            loaded += 1

    print(f"Loaded {loaded} expenses from '{filename}'. Skipped {skipped} malformed rows.\n")
except Exception as exc:
    print(f"Error loading expenses from '{filename}': {exc}\n")
```

```python
# menu & main loop
def print_menu():
    print("Expense Tracker Menu:")
    print(" 1) Add expense")
    print(" 2) View expenses")
    print(" 3) Track budget")
    print(" 4) Save expenses")
    print(" 5) Exit (saves and quits)")
```

```python
def main():
    load_expenses_from_csv(EXPENSES_FILE)
    try:
        while True:
            print_menu()
            choice = input("Choose an option (1-5): ").strip()
            if choice == "1":
                add_expense()
            elif choice == "2":
                view_expenses()
            elif choice == "3":
                # Ask user if they want to just view comparison or set budget first
                track_budget()
            elif choice == "4":
                save_expenses_to_csv(EXPENSES_FILE)
            elif choice == "5":
                save_expenses_to_csv(EXPENSES_FILE)
                print("Goodbye! Expenses saved. Exiting.")
                break
            else:
                print("Invalid option. Please choose a number between 1 and 5.\n")
    except KeyboardInterrupt:
        print("\nDetected keyboard interrupt. Saving expenses before exit...")
        save_expenses_to_csv(EXPENSES_FILE)
        print("Exited via keyboard interrupt. Goodbye.")
```

```
        elif choice == "2":
            view_expenses()
        elif choice == "3":
            # Ask user if they want to just view comparison or set budget first
            track_budget()
        elif choice == "4":
            save_expenses_to_csv(EXPENSES_FILE)
        elif choice == "5":
            save_expenses_to_csv(EXPENSES_FILE)
            print("Goodbye! Expenses saved. Exiting.")
            break
        else:
            print("Invalid option. Please choose a number between 1 and 5.\n")
    except KeyboardInterrupt:
        print("\nDetected keyboard interrupt. Saving expenses before exit...")
        save_expenses_to_csv(EXPENSES_FILE)
        print("Exited via keyboard interrupt. Goodbye.")


if __name__ == "__main__":
    main()
```

```
Expense Tracker Menu:
 1) Add expense
 2) View expenses
 3) Track budget
 4) Save expenses
 5) Exit (saves and quits)
Choose an option (1-5):
```

[ ]:

```
Expense Tracker Menu:
 1) Add expense
 2) View expenses
 3) Track budget
 4) Save expenses
 5) Exit (saves and quits)
Choose an option (1-5):  1

Add an expense (enter 'q' at any prompt to cancel):
Date (YYYY-MM-DD): ↑↓ for history. Search history with c-↑/c-↓
```

```
[ ]:
```

```
Expense Tracker Menu:
 1) Add expense
 2) View expenses
 3) Track budget
 4) Save expenses
 5) Exit (saves and quits)
Choose an option (1-5):  1

Add an expense (enter 'q' at any prompt to cancel):
 Date (YYYY-MM-DD):  2025-05-05
 Category (e.g. Food, Travel):  food
 Amount (e.g. 15.50):  500
Description (brief): ↑↓ for history. Search history with c-↑/c-↓
```

[ ]:

```
Expense Tracker Menu:
 1) Add expense
 2) View expenses
 3) Track budget
 4) Save expenses
 5) Exit (saves and quits)
Choose an option (1-5):  1

Add an expense (enter 'q' at any prompt to cancel):
 Date (YYYY-MM-DD):  2025-05-05
 Category (e.g. Food, Travel):  food
 Amount (e.g. 15.50):  500
 Description (brief):  b'day expense
Expense added successfully:
  {'date': '2025-05-05', 'category': 'food', 'amount': 500.0, 'description': "b'day expense"}

Expense Tracker Menu:
 1) Add expense
 2) View expenses
 3) Track budget
 4) Save expenses
 5) Exit (saves and quits)
Choose an option (1-5): ↑↓ for history. Search history with c-↑/c-↓
```

```
No monthly budget set yet.
Would you like to set it now? (y/n):  y
Enter your monthly budget amount (e.g. 1000.00):  5000
Monthly budget set to: 5000.00


Total expenses so far: 500.00
Monthly budget: 5000.00
You have 4500.00 left for the month.
```

```
Expenses saved to 'expenses.csv'.

Expense Tracker Menu:
 1) Add expense
 2) View expenses
 3) Track budget
 4) Save expenses
 5) Exit (saves and quits)
Choose an option (1-5):  5
Expenses saved to 'expenses.csv'.

Goodbye! Expenses saved. Exiting.
```